

Lab 11

Samuel Abrão e João Genaro

30 de outubro de 2025

Introdução

Este relatório descreve a implementação e a análise de uma aplicação de rede baseada em TCP no modelo cliente-servidor. O objetivo foi construir um servidor de eco que recebesse dados de um cliente, devolvesse exatamente o que recebeu e, com isso, permitir a observação prática do estabelecimento de conexão, da troca de dados e do encerramento de uma sessão TCP. Além de confirmar o funcionamento lógico dos programas, investiguei o tráfego usando o Wireshark para relacionar o comportamento da aplicação aos pacotes e flags no nível de transporte.

Objetivos

O trabalho buscou consolidar conceitos fundamentais de TCP por meio de um experimento simples e rastreável. Em termos aplicados, os objetivos foram preparar um servidor que escutasse no endereço de loopback em uma porta fixa e um cliente que enviasse mensagens a esse servidor, recebesse o eco correspondente e encerrasse a sessão ao enviar a palavra-chave definida. Em termos de análise, o objetivo foi identificar no traço capturado pelo Wireshark as três fases essenciais: o handshake de três vias para o estabelecimento da conexão, os segmentos que carregam a carga útil durante a conversação e a sequência de finalização da conexão com as trocas adequadas de flags.

Ambiente e metodologia

Os testes foram executados localmente, conectando cliente e servidor via 127.0.0.1 e utilizando a porta 12345 para facilitar a filtragem e a repetibilidade. O servidor foi iniciado antes do cliente para assegurar que o *listen* estivesse pronto no momento do *connect*. Para a análise de rede, foi utilizado o Wireshark monitorando a interface de loopback, aplicando o filtro `tcp.port == 12345` para focar unicamente na sessão do experimento. A captura foi iniciada com o servidor já em escuta e o cliente ainda não conectado, de modo a registrar desde os primeiros pacotes de estabelecimento da conexão até o encerramento completo.

Descrição da implementação

A aplicação é formada por dois programas. O primeiro é o servidor, que cria um socket TCP, associa-se ao endereço de loopback e à porta definida e entra em modo de escuta.

Ao aceitar a conexão do cliente, o servidor passa a receber *buffers* de bytes, converter para string e enviar de volta exatamente a mesma sequência de bytes, caracterizando um serviço de eco. O servidor permanece em laço até detectar a mensagem de término, definida como a palavra *sair*, que também é ecoada antes do encerramento. O segundo programa é o cliente, que estabelece a conexão TCP com o servidor no mesmo endereço e porta. Em seguida, ele entra em laço de leitura do teclado, envia o que foi digitado e aguarda a resposta do servidor para exibição imediata. Quando o usuário digita a palavra *sair*, o cliente envia essa mensagem, recebe o eco e encerra a execução, o que aciona a sequência de finalização da conexão no nível do transporte.

Procedimento de execução

Inicialmente, o servidor foi iniciado e verificou-se que a porta estava em escuta no endereço local. Em seguida, foi aberta a captura no Wireshark com o filtro preparado, garantindo que somente os pacotes relevantes fossem visíveis. Depois, o cliente foi executado e a primeira interação consistiu em enviar uma saudação curta para verificar o retorno imediato do eco. Foram executadas algumas variações como envio de palavras simples, linhas com espaços e pequenas sequências, observando sistematicamente o reflexo no Wireshark. Por fim, digitou-se *sair* para provocar o encerramento protocolar e capturar a respectiva troca de pacotes de finalização.

Observações no Wireshark

No momento em que o cliente iniciou a conexão com o servidor, observou-se o *three-way handshake* característico do TCP. O primeiro segmento foi enviado pelo cliente com a flag SYN para solicitar a sincronização de números de sequência, seguido pela resposta do servidor com as flags SYN e ACK, confirmando a recepção e propondo a sua própria sincronização. Em seguida, o cliente enviou um segmento apenas com a flag ACK para concluir o estabelecimento da conexão. Uma vez estabelecida a sessão, cada mensagem digitada no cliente resultou em um segmento com carga útil chegando ao servidor, geralmente marcado no Wireshark com PSH+ACK, indicando que os dados deveriam ser imediatamente empurrados à aplicação receptora sem aguardar preenchimento de *buffer*. A resposta do servidor, por sua vez, também apareceu como um segmento com dados acompanhado de ACK, seguida pelos ACKs de confirmação a cada entrega. Ao enviar a palavra *sair*, registrou-se a fase de encerramento com a troca dos segmentos FIN e ACK entre as pontas, evidenciando o fechamento ordenado do canal duplex. Todo o tráfego ficou visível com clareza usando a filtragem pela porta definida e a interface de loopback, o que eliminou ruído de outras aplicações.

Discussão

A experiência confirmou, na prática, a relação direta entre o comportamento da aplicação e as transições de estado do TCP. O serviço de eco permitiu testar rapidamente a confiabilidade do fluxo e observar a entrega em ordem, enquanto o uso do PSH destacou a entrega imediata à aplicação sem depender de um *buffering* mais agressivo. O uso de uma palavra-chave explícita para término facilitou a predição do padrão de pacotes de finalização, permitindo conferir no traço a sequência completa de fechamento de conexão.

No desenho do servidor, a decisão de tratar uma única conexão simplificou a análise e evitou concorrência, o que é apropriado para fins didáticos e para que o traço de rede se mantenha limpo. Em um cenário de produção, evoluiríamos o servidor para suportar múltiplos clientes simultâneos com *threads* ou mecanismos assíncronos, além de acrescentar validações robustas, tratamento de exceções e registros de atividade. Ainda assim, para os propósitos do laboratório, a implementação mínima demonstrou os pilares do TCP de forma transparente.

Conclusão

O laboratório atingiu seus objetivos ao integrar implementação prática e inspeção protocolar. A dupla cliente–servidor demonstrou a construção de um serviço de eco funcional no transporte TCP e a análise com o Wireshark consolidou conceitos essenciais como o handshake de três vias, a sinalização PSH+ACK durante a troca de dados e o encerramento ordenado com FIN/ACK. O uso de 127.0.0.1 e da porta 12345 simplificou a configuração e a filtragem, tornando a observação do tráfego direta e reproduzível. Como próximos passos, recomenda-se experimentar mensagens maiores, diferentes padrões de entrada e variações de *buffer*, além de estender o servidor para múltiplos clientes, explorando as implicações de concorrência e controle de fluxo.

Referências

Material do laboratório sobre sockets TCP, interação cliente–servidor e observação com Wireshark, incluindo orientações para uso do endereço de loopback, filtragem pela porta de testes, identificação do handshake, visualização de pacotes PSH+ACK durante a conversação e verificação do encerramento com FIN/ACK.